# Introduction to VLSI System Design Final Project: Four-tap Fourier Transform

This circuit will implement a fast Fourier transform algorithm. Given four time-domain complex numbers, the circuit outputs four complex, frequency-domain Fourier coefficients. This is the building block for larger FFTs that can enable image noise removal, speech recognition, and fast convolution/correlation. We will be implementing the general DFT algorithm for the first N terms:

$$X[k] = \frac{1}{N}\sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi}{N}kn}$$

So in our circuit, we'll find the first four terms:

$$X[k] = \frac{1}{4}\sum_{n=0}^{3} x[n]e^{-j\frac{2\pi}{4}kn}$$

Here $e^{-j\frac{2\pi}{4}kn}$ is represented by the twiddle factor: $W_4^n$.

The 4-input butterfly can thus be achieved with simple multiplication, subtraction, addition, and division.

$$W_4^0 = 1 = (1, 0)$$
$$W_4^1 = -j = (0, -1)$$
$$W_4^2 = -1 = (-1, 0)$$
$$W_4^3 = j = (0, 1)$$

In a four-tap fourier transform, the DFT can be expressed as the following expressions:[1]

Stage 1:

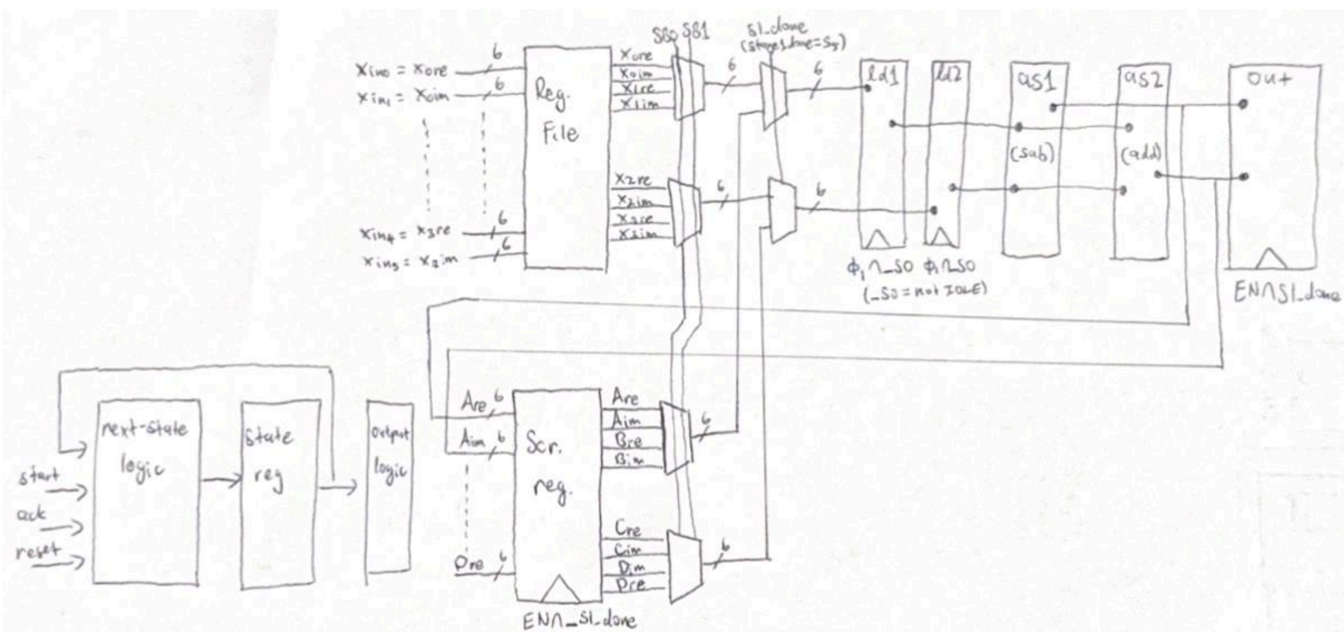
Butterfly 1:
$A_{REAL} = x_{0,re} + x_{2,re}$
$A_{IMAG} = x_{0,imag} + x_{2,imag}$
$B_{REAL} = x_{0,re} - x_{2,re}$
$B_{IMAG} = x_{0,imag} - x_{2,imag}$
$C_{REAL} = x_{1,re} + x_{3,re}$
$C_{IMAG} = x_{1,imag} + x_{3,imag}$
$D_{REAL} = x_{1,re} - x_{3,re}$
$D_{IMAG} = x_{1,imag} - x_{3,imag}$

[1] https://www.alwayslearn.com/DFT%20and%20FFT%20Tutorial/DFTandFFT_TheDFT.html

Stage 2:

$F_{REAL}(0) = A_{REAL} + C_{REAL}$
$F_{IMAG}(0) = A_{IMAG} + C_{IMAG}$
$F_{REAL}(1) = B_{REAL} + D_{IMAG}$
$F_{IMAG}(1) = B_{IMAG} - D_{REAL}$
$F_{REAL}(2) = A_{REAL} - C_{REAL}$
$F_{IMAG}(2) = A_{IMAG} - C_{IMAG}$
$F_{REAL}(3) = B_{REAL} - D_{IMAG}$
$F_{IMAG}(3) = B_{IMAG} + D_{REAL}$

In our design, the circuit takes in a six-bit number as input. It can accept integer inputs from 0-15. We intend to tape out our design, but further testing and verification will be required to do so.




Our circuit above will take in 6 bits of input and load the inputted values into the register file ($x_{0,re}$ - $x_{3,imag}$). These values are outputted from the register file into two 4 to 1 muxes to select the appropriate pairs for stage 1 calculations. These are then fed into two 2 to 1 muxes to select the data required for 1st stage computations vs. 2nd stage computations (the datapath is reused to optimize spacing). In the first stage, the chosen pairs are then subtracted and added simultaneously and stored in an output latch. The calculations from the two adders are also stored into the scratch register for stage 2. Once $A_{REAL}$ - $D_{Imaginary}$ are calculated and stored into the scratch register, the steps of stage 1 are repeated to calculate $F_{REAL}(0)$ to $F_{IMAG}(3)$, and the final result is outputted.

Inputs:

1. **Reset**
2. **Phi0**
3. **Phi1**
4. **GND**
5. **GND**
6. **Vdd**
7. **Vdd**
8. START
9. ACK
10. WA0
11. WA1
12. WA2
13. xin0
14. xin1
15. xin2
16. xin3
17. xin4
18. xin5
19. REN
20. RA0
21. RA1

Outputs

22. outim_0
23. outim_1
24. outim_2
25. outim_3
26. outim_4
27. outim_5
28. outre_0
29. outre_1
30. outre_2
31. outre_3
32. outre_4
33. outre_5

Instructions
Inputs:
Use the table below to load in each 6-bit input one at a time by setting the corresponding write address and ensuring WEN is HIGH:

| WEN | WA0 | WA1 | WA2 | Input |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | $x_{3,im}$ |
| 1 | 1 | 1 | 0 | $x_{1,im}$ |
| 1 | 1 | 0 | 1 | $x_{3,re}$ |
| 1 | 1 | 0 | 0 | $x_{1,re}$ |
| 1 | 0 | 1 | 1 | $x_{2,im}$ |
| 1 | 0 | 1 | 0 | $x_{0,im}$ |
| 1 | 0 | 0 | 1 | $x_{2,re}$ |
| 1 | 0 | 0 | 0 | $x_{0,re}$ |

To start, set START to HIGH and begin the clock

Once the computations are complete and DONE outputs HIGH, use the table below to read out 2 6-bit outputs (the real and imaginary components of the given result) at a time by setting the corresponding read address and ensuring REN is HIGH.

| REN | RA0 | RA1 | Output |
|---|---|---|---|
| 1 | 0 | 0 | F(0) |
| 1 | 0 | 1 | F(1) |
| 1 | 1 | 0 | F(2) |
| 1 | 1 | 1 | F(3) |